



Practical 8 : Binary Tree Traversals and Binary Search Tree Insertion

→ Practical Aim

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree.

Problem 1

A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each. Describe how your solution for level-by-level printing differs from the other three. What additional structure did you need to use, and why does the same recursive approach that works for the other three not directly apply here?

Source Code:

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/1ed2896fd0cc4e1a7f4eb4c2dcfea56bde57f34a/PRACTICAL%208

OUTPUT

```
"C:\Users\diyat\OneDrive\Doc" × + v
Inorder: 4 2 5 1 6 3 7
Preorder: 1 2 4 5 3 6 7
Postorder: 4 5 2 6 7 3 1
Level Order: 1 2 3 4 5 6 7
Process returned 0 (0x0)   execution time : 0.163 s
Press any key to continue.
|
```

Conclusion:

In this practical, I implemented inorder, preorder, postorder, and level-order traversal of a binary tree. I understood how recursive traversal works for the first three methods and how a queue is used for level-order traversal.

Problem 2

A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete. Describe how your solution decides where to place each new book code. What property of the final inorder sequence tells you whether all insertions were performed correctly?

Source Code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/1ed2896fd0cc4e1a7f4eb4c2dcfea56bde57f34a/PRACTICAL%208

OUTPUT

```
"C:\Users\diyat\OneDrive\Do... × + v
Enter number of books: 7
Enter book codes:
50 30 70 20 40 60 80

Inorder sequence: 20 30 40 50 60 70 80
Process returned 0 (0x0)   execution time : 25.425 s
Press any key to continue.
```

Conclusion:

In this practical, I implemented insertion of book codes into a Binary Search Tree. I understood that smaller values are placed on the left and larger values on the right. The inorder traversal gives the values in ascending order, which helps to check whether the BST is created correctly.

Challenges Faced

Problem 1: Understanding the difference between inorder, preorder, postorder, and level-order traversal.

Solution: I understood the order in which each traversal visits the nodes and implemented them separately.

Problem 2: Deciding whether a new book code should be placed on the left or right side.

Solution: I compared the new value with the current node. Smaller values go left and larger values go right.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

Answer: Preorder traversal gives the root first, but it is not enough to reconstruct the complete

original binary tree because we do not know which nodes are on the left and right sides. We need additional information such as the inorder traversal. Preorder + inorder can uniquely reconstruct a binary tree when the node values are unique.

2. [Problem 2] The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both? How would you isolate which one?

Answer: If the inorder output of a BST is not sorted, there may be an error in the insertion logic, the inorder traversal logic, or both. To isolate the error, first check the insertion by drawing or checking the BST structure and verifying that smaller values are on the left and larger values are on the right. Then test the inorder traversal on a simple known tree. If the tree structure is correct but the output is not sorted, the traversal logic is incorrect. If the traversal works correctly on a known tree but the BST is not ordered, the insertion logic is incorrect.